

Plano de implementação — coleta confiável de KPIs e VIPs

1. Objetivo

Corrigir o pipeline de coleta para que o SmartEvents processe os contratos atuais do iManager, calcule corretamente os KPIs 4G e 5G, consuma os traces de VIP de forma incremental e nunca apresente uma coleta como saudável quando houve falha, perda de cobertura, dados inválidos ou ausência prolongada de medições.

O trabalho deve ser dividido em quatro fases. A divisão é necessária porque a mudança atravessa autenticação, contratos HTTP externos, cálculo de métricas, persistência, agendamento, API, interface e testes com VPN. Cada fase deixa uma entrega verificável e reduz o risco de misturar falhas de infraestrutura com erros de fórmula ou de apresentação.

2. Evidências e decisões que orientam o plano

- `novo-log-11-08-2026-13h50.log` mostra ciclos de Monitoring com dezenas de objetos, mas zero medições, além de falhas HTTP 500 repetidas em `filter-by-cols`.
- `novo-log-11-08-2026-14h47.log` mostra renovações declaradas como bem-sucedidas seguidas imediatamente por HTTP 401 nos POSTs reais.
- O estado atual em `core/scheduler.py` registra `state="ok"` e atualiza `last_success` mesmo quando o coletor retorna uma lista vazia após capturar uma exceção.
- O Monitoring atual devolve contadores brutos em `counterRes`, enquanto `counterExpRes` está vazio. O parser em `core/collector.py` procura principalmente nomes de métricas já calculadas.
- O navegador envia o último `preExecTime` confirmado pelo servidor e recebe um novo `execTime`. O coletor atual envia o relógio local como cursor e descarta os cursors retornados.
- O fluxo atual de VIP usa `filter-by-cols`; `novo-curl-vip.md` mostra o navegador consumindo `query/subscribe-result` com `lastSerialNo`.
- A captura de VIP usa a task 2073, enquanto os logs do evento usam a task 2070. O contrato precisa ser validado com a mesma task configurada para o VIP do evento.
- A task 747 capturada no Monitoring contém objetos LTE. O evento de teste possui também células 5G, que não aparecem nessa resposta; o suporte 5G depende de localizar e configurar a task PM correspondente.
- `kpi_measurements` ainda não possui deduplicação. `vip_measurements` deduplica por nome e timestamp, mas o parser remove os milissegundos e pode colidir quando há mais de um relatório no mesmo segundo.
- Todos os KPIs dos arquivos `Fórmulas_4G_Monitoring(1).txt` e `Fórmulas_5G_Monitoring.txt` estão no escopo. Divisão por zero, contador ausente ou contador não confiável não deve produzir zero: deve produzir uma medição inválida com motivo observável.
- Alarmes entram apenas nas mudanças compartilhadas de sessão, resultado de coleta e status. A consulta funcional e os filtros de alarmes não serão redesenhados nesta implementação.

3. Contratos transversais

3.1 Resultado explícito de coleta

Todo coletor deve devolver um resultado estruturado, em vez de usar `[]` para representar situações diferentes. O resultado deve conter:

- estado do ciclo: `data`, `empty`, `partial`, `error` ou `auth_required`;
- medições válidas produzidas;
- cursores confirmados pelo servidor, ainda não persistidos;
- quantidade recebida, calculada, inválida, duplicada e efetivamente inserida;
- cobertura dos alvos configurados, como células mapeadas e VIPs com dados;
- erros por etapa e uma causa curta apropriada para a interface;
- horário do dado mais recente retornado pelo sistema externo.

O agendador deve manter separadamente `last_attempt_at`, `last_cycle_ok_at` e `last_data_at`. Um ciclo vazio válido pode atualizar `last_cycle_ok_at`, mas não `last_data_at`. Erros de autenticação, HTTP, contrato ou parsing não podem atualizar nenhum campo de sucesso.

3.2 Cursores e persistência idempotente

- O cursor enviado deve ser sempre o último cursor confirmado pelo iManager, nunca o relógio local.
- Um cursor só pode ser salvo depois que o lote correspondente tiver sido persistido.
- Se o processo cair entre a persistência e a atualização do cursor, o servidor poderá reenviar o lote; os índices únicos devem tornar o replay seguro.
- Cursores devem ser isolados por evento, OSS, tipo de coletor, task e objeto quando aplicável.
- O valor zero é uma medição válida. Somente ausência, invalidade matemática ou falta de confiabilidade deve omitir a medição.

3.3 Semântica visual de saúde

A interface deve diferenciar:

- **Coletando**: ciclo em andamento;
- **Com dados**: ciclo válido que inseriu ou confirmou dados novos;
- **Sem novidade**: requisição válida, mas nenhum dado novo desde o cursor anterior;
- **Parcial**: somente parte das células/VIPs configurados foi processada;
- **Erro**: falha de conexão, HTTP, contrato ou parsing;
- **Reautenticação necessária**: a sessão não pôde ser recuperada automaticamente;
- **Desatualizado**: `last_data_at` ultrapassou o limite esperado para o coletor.

O envelope `ok` de `Api.get_collection_status()` deve significar apenas que a chamada local foi atendida. A saúde dos coletores deve estar em `overall_state` e nos estados individuais.

4. Catálogo de KPIs a implementar

O código deve usar identificadores canônicos, mantendo as fórmulas fornecidas como fonte de verdade. Fórmulas comuns podem compartilhar o mesmo identificador entre 4G e 5G; a tecnologia determina quais contadores alimentam o cálculo.

Tecnologia	KPI	Identificador sugerido	Unidade	Agregação no site
4G	Acessibilidade de Dados	accessibility	%	recalcular com contadores somados
4G	Availability	availability	%	recalcular com duração e período
4G	Drop Dados	drop_rate	%	recalcular com contadores somados
4G	DL PRB Utility	utilization_dl	%	recalcular ponderando PRBs disponíveis
4G	UL PRB Utility	utilization_ul	%	recalcular ponderando PRBs disponíveis
4G	Interferência	interference_ul	dBm	média das células válidas
4G	Throughput DL	throughput_dl	Mbit/s	soma dos resultados por célula
4G	Throughput UL	throughput_ul	Mbit/s	soma dos resultados por célula
4G	UE médio	user_count	usuários	soma das médias por célula
4G	Wireless RTT	ran_rtt	ms	média das células válidas
4G	Terrestrial RTT	terrestrial_rtt	ms	média das células válidas
5G SA	Acessibilidade considerando RRC Inactive	accessibility	%	recalcular com contadores somados
5G SA	Drop considerando RRC Inactive	drop_rate	%	recalcular com contadores somados
5G	DL PRB Utility	utilization_dl	%	recalcular ponderando PRBs disponíveis

Tecnologia	KPI	Identificador sugerido	Unidade	Agregação no site
5G	UL PRB Utility	utilization_ul	%	recalcular ponderando PRBs disponíveis
5G	Throughput DL	throughput_dl	validar no OSS	soma dos resultados por célula
5G	Throughput UL	throughput_ul	validar no OSS	soma dos resultados por célula
5G SA	Downlink Traffic Volume	traffic_volume_dl_sa	validar no OSS	soma
5G NSA	Downlink Traffic Volume	traffic_volume_dl_nsa	validar no OSS	soma
5G SA	Uplink Traffic Volume	traffic_volume_ul_sa	validar no OSS	soma
5G NSA	Uplink Traffic Volume	traffic_volume_ul_nsa	validar no OSS	soma
5G	User Médio	user_count	usuários	soma das médias por célula
5G	Availability	availability	%	duração disponível / células / período
5G	UL Interference Médio	interference_ul	unidade do contador, esperada dBm	média das células válidas

Os atuais `traffic_volume_dl` e `traffic_volume_ul` de 4G devem continuar disponíveis por compatibilidade. Como eles não aparecem no novo arquivo de fórmulas 4G, sua conversão deve ser caracterizada e documentada, sem misturá-los às fórmulas novas ou alterar silenciosamente a unidade.

As unidades de throughput e volume 5G devem ser comparadas com a tela do iManager usando uma captura real. Até essa comparação passar, esses KPIs podem existir nos testes por fixture, mas não devem ser considerados liberados para produção.

5. Fase 1 — resultado confiável, sessão e status operacional

Explicação simples

Esta fase elimina o falso "OK". Ela cria um contrato único para que qualquer falha, vazio legítimo, resultado parcial ou problema de autenticação chegue corretamente do coletor até o painel.

Escopo de código

- Criar `core/collection_result.py` com o resultado estruturado e os diagnósticos compartilhados pelos coletores.
- Alterar as interfaces de `BaseCollector`, `MockCollector`, `CsvCollector` e `HttpCollector` em `core/collector.py` para devolver o novo resultado.
- Remover blocos que capturam erro e retornam `[]` sem preservar a causa. Erros recuperáveis devem virar estados explícitos; erros inesperados devem continuar chegando ao agendador.
- Atualizar `core/scheduler.py` para:
 - não marcar vazio, parcial ou erro como `ok`;
 - manter horários de tentativa, ciclo válido e último dado separadamente;
 - calcular estado desatualizado com base no intervalo do coletor;
 - contabilizar registros recebidos, inválidos, duplicados e inseridos;
 - preservar o último erro até existir um ciclo comprovadamente saudável.
- Corrigir `core/session_renew.py` e a renovação em `core/collector.py` para:
 - não reutilizar task IDs, objetos ou tokens antigos como se tivessem sido recapturados;
 - gravar `session.json` de forma atômica somente após cookies, `roarand` e metadados necessários estarem coerentes;
 - reconstruir a `requests.Session` após a renovação;
 - considerar a renovação efetiva somente quando a chamada original que falhou funcionar no `retry`;
 - manter estados separados para Monitoring/Alarmes e Trace, mesmo que compartilhem cookies;
 - transformar 401/403/redirect para SSO em `auth_required`, e timeout/500/contrato inválido em `error`.
- Atualizar `api/api.py` para devolver o novo contrato de status e um `overall_state` calculado sem esconder falhas individuais.
- Atualizar `frontend/js/bridge.js` com cenários mock reproduzíveis para todos os estados visuais.

Escopo de interface

- Atualizar o modal de coleta em `frontend/js/app.js` e os estilos em `frontend/css/main.css`.
- Exibir `Última tentativa`, `Último ciclo válido` e `Último dado`, em vez de um único horário ambíguo.
- Exibir a cobertura do ciclo: células mapeadas/esperadas, VIPs com dados/configurados e medições válidas/inválidas.
- Usar cores e textos distintos para vazio, parcial, erro, autenticação e desatualização.
- Não mostrar a sessão como "OK" somente porque não há alerta interativo; mostrar a situação real de cada módulo.
- Escapar todo texto de erro antes de inseri-lo no HTML, mantendo o padrão já usado por `_esc`.

Escopo de testes

- Criar `tests/test_collection_result.py` para validar a classificação de resultados.
- Ampliar `tests/test_scheduler.py` com cenários de dados, vazio legítimo, parcial, 401, 500, timeout, exceção de parsing e estado desatualizado.
- Ampliar `tests/test_api.py` para validar o novo contrato e garantir que `ok: true` não transforme um coletor com erro em saudável.
- Ampliar `tests/test_collector.py` com sessões falsas que comprovem o retry pós-renovação e a preservação da causa.
- Criar `tests/test_frontend_collection_ui.py` usando o Playwright já instalado para abrir o frontend com os mocks e conferir texto e classes dos estados principais, sem introduzir um segundo ecossistema de testes JavaScript.

Como validar e testar

Testes automatizados

1. Executar os testes específicos de resultado, agendador, API e interface.
2. Executar a suíte offline completa com `python -m pytest -m "not vpn"`.
3. Forçar uma resposta 401 seguida de renovação e retry válido; somente o retry deve limpar o erro.
4. Forçar uma renovação que retorna sucesso no subprocesso, mas falha no POST seguinte; o estado deve permanecer `auth_required` ou `error`.

Validação visual

1. Abrir o frontend em modo de desenvolvimento e alternar os mocks entre `data`, `empty`, `partial`, `error`, `auth_required` e `stale`.
2. Confirmar que zero medições com erro nunca aparece como "OK".
3. Confirmar que "sem novidade" não parece falha e preserva o horário do último dado.
4. Confirmar legibilidade de mensagens longas e ausência de quebra do modal em resolução semelhante à captura fornecida.

Sugestão de commit: `feat: tornar o status da coleta confiável`

6. Fase 2 — coleta e cálculo dos KPIs 4G e 5G

Explicação simples

Esta fase adapta o Monitoring ao retorno atual de contadores brutos, aplica todas as fórmulas fornecidas e garante que cada ciclo avance pelo cursor do servidor sem duplicar ou pular dados.

Escopo de código

- Criar `core/kpi_formulas.py` como catálogo declarativo de métricas. Cada definição deve registrar tecnologia, nome, unidade, contadores obrigatórios, regra de cálculo e agregação de site.
- Implementar as fórmulas como funções controladas no código, sem avaliador genérico de expressões de texto.
- Converter `counterRes` em um mapa de contadores por objeto e aplicar somente as fórmulas compatíveis com a tecnologia da célula.
- Tratar como inválida qualquer fórmula com contador ausente, valor não numérico, `reliable` diferente de confiável ou denominador zero. Registrar métrica, célula e motivo nos diagnósticos, sem inserir linha com zero artificial.
- Usar `period` da resposta como `GP / SP` nas fórmulas de disponibilidade. Não usar o intervalo local do agendador como substituto.
- Normalizar a tecnologia das células para 4G/5G a partir do campo `tech` quando presente e, por compatibilidade, do prefixo do ID da célula. Rejeitar mapeamentos ambíguos.
- Substituir o matching amplo por substring por uma normalização determinística de nome, site e tecnologia. Células não mapeadas devem entrar na cobertura parcial com seus nomes.
- Alterar a configuração de integração para aceitar tasks PM por tecnologia, mantendo `pm_task_id` como fallback compatível para 4G. A forma sugerida é uma coleção `pm_tasks` com `task_id` e `tech`.
- Atualizar `core/session_renew.py` para capturar múltiplas tasks e seus `objNo`, sem reduzir Monitoring a um único `task_id`.
- Separar descoberta de objetos de consumo incremental:
 - fazer descoberta inicial por task;
 - continuar coletando objetos já mapeados mesmo se outra tecnologia ainda estiver sem task;
 - não repetir a consulta aberta que fez os logs crescerem de 56 para 168 objetos.
- Enviar o último `preExecTime` confirmado por task/objeto e aproveitar `execTime` e `objNoExecTimes` retornados como próximos checkpoints.
- Adicionar em `core/database.py` uma tabela de checkpoints isolada por evento, OSS, coletor, task e chave de objeto.
- Migrar `kpi_measurements` com deduplicação e índice único por evento, site, célula/escopo, timestamp e métrica. O insert deve ignorar replay e devolver contagens de inseridos/duplicados.
- Persistir também o agregado correto de site calculado a partir dos contadores brutos do mesmo timestamp. Percentuais compostos não devem ser obtidos pela média simples de percentuais de células.
- Adicionar um campo de escopo compatível com linhas de célula e de site, atualizando as consultas em `core/database.py` e `api/api.py` para não misturá-las.
- Centralizar em `api/api.py` a leitura do catálogo de métricas e sua agregação, removendo regras conflitantes hoje espalhadas entre gráfico, lista e tooltip.
- Preservar os volumes 4G existentes como métricas legadas com testes de caracterização.
- Criar fixtures sanitizadas, sem cookies ou tokens, em:

- `tests/fixtures/monitoring_4g_raw.json` com uma amostra mínima derivada de `novo-curl-monitoring.md`;
- `tests/fixtures/monitoring_5g_raw.json` com os contadores necessários para todas as fórmulas 5G, substituída por uma captura real assim que a task 5G for localizada.

Escopo de interface

- Atualizar `frontend/index.html` e `frontend/js/kpi.js` para carregar o catálogo disponível em vez de manter uma lista fixa incompleta.
- Agrupar as opções em métricas comuns, adicionais 4G e volumes 5G SA/NSA.
- Mostrar unidade, tecnologia e regra de "Site completo" conforme os metadados do KPI.
- Ocultar ou desabilitar métricas que não existem para o site selecionado, sem apresentar zero.
- Diferenciar `0` válido de ausência de ponto. Fórmulas inválidas devem aparecer como lacuna no gráfico e como motivo no status de coleta.
- Mostrar um badge 4G/5G nas células para que o operador entenda qual fórmula foi aplicada.
- Manter os gráficos por célula e usar a linha agregada persistida para "Site completo".

Escopo de testes

- Criar `tests/test_kpi_formulas.py` com um vetor conhecido para cada fórmula 4G e 5G.
- Para cada fórmula com divisão, cobrir denominador zero, contador ausente, valor não numérico e confiabilidade inválida.
- Validar o uso de `period` para disponibilidade e as conversões de unidade explicitamente definidas.
- Criar testes de parsing em `tests/test_kpi_monitoring.py` com `counterRes`, `counterExpRes` vazio, múltiplos resultados, 4G e 5G, objeto desconhecido e nome ambíguo.
- Testar bootstrap, segundo ciclo com cursor, reinício com checkpoint persistido e replay do mesmo payload.
- Ampliar `tests/test_database.py` para cobrir migração, índice único, replay e linhas de escopo `CELL / SITE`.
- Ampliar `tests/test_api.py` para catálogo, unidades, agregados e séries sem duplicatas.
- Ampliar o teste de interface para conferir o seletor dinâmico, os badges, o zero válido e as lacunas de dados inválidos.

Como validar e testar

Testes automatizados

1. Executar `tests/test_kpi_formulas.py` , `tests/test_kpi_monitoring.py` , `tests/test_database.py` e `tests/test_api.py` .
2. Processar a fixture 4G duas vezes: a primeira execução deve inserir o conjunto esperado e a segunda deve resultar apenas em duplicatas ignoradas.
3. Simular dois ciclos em sequência e conferir que o segundo request usa exatamente o `execTime` / `preExecTime` devolvido no primeiro.
4. Confirmar que uma célula 5G sem task produz cobertura parcial, não estado saudável com zero medições.

Validação visual

1. Carregar um evento mock com sites 4G e 5G.
2. Confirmar que cada site oferece somente KPIs aplicáveis e com unidades corretas.
3. Conferir valores de célula e "Site completo" contra cálculos manuais das fixtures.
4. Confirmar que um KPI matematicamente inválido cria uma lacuna e nunca um ponto zero.
5. Comparar os valores 4G ao resultado do iManager para o mesmo `execTime` .
6. Localizar a task PM 5G, capturar um ciclo real e comparar todos os KPIs 5G com a tela do iManager. Throughput e volume 5G só passam pelo gate de liberação após essa comparação de unidade.

Sugestão de commit: `feat: calcular e persistir KPIs 4G e 5G`

7. Fase 3 — coleta incremental de VIPs pelo contrato atual

Explicação simples

Esta fase substitui o endpoint que falha por 500/401 pelo consumo incremental observado no navegador, preservando a ordem dos traces e garantindo que a task consultada pertence ao VIP configurado.

Pré-requisito de contrato

Antes da implementação, capturar no Network do navegador o fluxo completo para a mesma task usada pelo evento, preferencialmente a task 2070 mostrada nos logs:

1. `pre-check` ;
2. inicialização que fornece `msgId` ;
3. `subscribe-result` com `lastSerialNo` ;
4. `msg-explain-info` de um `RRC_MEAS_RPRT` retornado nessa assinatura.

A captura atual da task 2073 prova o uso de `subscribe-result` , mas não prova a inicialização, a associação com o VIP Baldin nem a forma correta de referenciar a linha no decode. A implementação não deve adivinhar esses campos.

Escopo de código

- Reescrever o fluxo de VIP em `core/collector.py` para usar a inicialização confirmada e `query/subscribe-result`.
- Remover `filter-by-cols` do caminho normal. Se o novo contrato falhar, devolver erro explícito em vez de cair silenciosamente no endpoint antigo.
- Persistir `lastSerialNo` por evento, OSS e task usando a infraestrutura de checkpoints da fase 2.
- Processar mensagens em ordem de serial e filtrar `RRC_MEAS_RPRT` no cliente.
- Avançar o cursor somente até o último serial tratado com segurança. Um decode transitório que falhar não pode fazer o coletor pular todos os seriais seguintes sem diagnóstico.
- Substituir o modelo `express/full` por consumo incremental limitado por lote. Se houver backlog, manter o estado parcial e continuar do mesmo cursor no ciclo seguinte.
- Reutilizar uma sessão coerente por worker e interromper o lote com `auth_required` quando qualquer etapa indicar expiração.
- Validar, antes da coleta, se todas as tasks configuradas existem e estão disponíveis. Expor task ausente, divergente ou parada no resultado do ciclo.
- Preservar milissegundos do timestamp do FARS.
- Alterar `vip_measurements` em `core/database.py` para armazenar `task_id` e `serial_no`.
- Substituir a deduplicação global por nome/timestamp por:
 - índice único de `(task_id, serial_no)` quando há serial;
 - fallback legado por `(vip_name, timestamp)` somente para registros antigos sem serial.
- Persistir o lote antes de confirmar `lastSerialNo`.
- Atualizar `tools/oss_validate.py` para exibir task, `msgId`, serial inicial/final, quantidade de mensagens, quantidade de `RRC_MEAS_RPRT`, decodes válidos e VIPs cobertos.
- Criar fixtures sanitizadas em `tests/fixtures/vip_subscribe_result.json` e `tests/fixtures/vip_msg_explain_info.json` a partir da captura completa.

Escopo de interface

- Atualizar o status de VIP em `frontend/js/app.js` para mostrar tasks válidas/configuradas, backlog, serial mais recente e quantidade de VIPs com dados.
- Trocar o rótulo de modo `expresso/completo` por `incremental` e, quando necessário, `processando backlog`.
- Em `frontend/js/vip.js`, diferenciar VIP sem mensagem nova, VIP com task inválida e VIP com erro de decode.
- Manter o último sinal válido no card, mas marcar visualmente quando ele estiver desatualizado; não apagar o histórico por causa de um ciclo vazio.

Escopo de testes

- Criar `tests/test_vip_subscription.py` cobrindo bootstrap, assinatura incremental, segundo ciclo, resposta sem mensagens, resposta sem `RRC_MEAS_RPRT` e cursor monotônico.
- Cobrir 401, 500, timeout, JSON inválido, `msgId` expirado e reinicialização da assinatura.
- Cobrir falha de decode no meio do lote e garantir que o checkpoint não pule o serial não tratado.
- Cobrir múltiplos relatórios no mesmo segundo e provar que `serial_no` preserva ambos.
- Cobrir task ausente, task diferente da configurada e múltiplos VIPs em paralelo.
- Ampliar `tests/test_database.py`, `tests/test_scheduler.py`, `tests/test_api.py` e o teste de interface com os novos campos.
- Corrigir `tests/test_http_vpn.py` para não aceitar uma lista vazia como sucesso quando existe um VIP ativo e uma task válida configurada.

Como validar e testar

Testes automatizados

1. Executar os testes de assinatura, banco, agendador e API.
2. Reproduzir a fixture duas vezes e confirmar que o segundo ciclo não cria registros adicionais.
3. Inserir duas mensagens no mesmo segundo com seriais distintos e confirmar que ambas persistem.
4. Simular falha no segundo decode de um lote; o próximo ciclo deve retomar do serial correto.

Validação visual e com VPN

1. Conferir no Network que o aplicativo usa `subscribe-result` e envia o último serial confirmado.
2. Rodar a coleta para a mesma task capturada e confirmar que o VIP correto recebe RSRP/RSRQ.
3. Comparar timestamp, serving cell, RSRP e RSRQ com a mensagem aberta no iManager.
4. Deixar ocorrerem vários ciclos e confirmar que o serial avança, o banco cresce apenas com mensagens novas e o card não fica falsamente saudável quando o trace para.
5. Forçar expiração da sessão e confirmar que o painel muda para reautenticação/erro até o retry real funcionar.

Sugestão de commit: `feat: coletar VIPs por assinatura incremental`

8. Fase 4 — validação ponta a ponta, teste prolongado e documentação

Explicação simples

Esta fase transforma os testes isolados em prova operacional. Ela valida vários ciclos reais, reinício, renovação de sessão, ausência temporária de dados e consistência entre banco, API e tela.

Escopo de código

- Evoluir `tools/oss_validate.py` para executar múltiplos ciclos e produzir um relatório final por coletor.
- Permitir que o validador verifique avanço de cursor, cobertura, registros inseridos, duplicatas, invalididades e idade do último dado.
- Fazer o processo retornar código diferente de zero quando:
 - um alvo configurado fica sem dados durante toda a janela esperada;
 - a sessão é declarada renovada, mas a chamada real continua falhando;
 - a cobertura de células/VIPs é parcial sem justificativa configurada;
 - o cursor não avança apesar de o iManager apresentar dados novos;
 - aparecem duplicatas ou fórmulas inválidas não explicadas.
- Atualizar `tests/test_http_vpn.py` para usar um evento de integração explicitamente selecionado, com tasks 4G, 5G e VIP conhecidas, em vez de considerar qualquer lista vazia aceitável.
- Adicionar um teste de reinício que persiste checkpoints, recria o coletor e confirma a continuidade.
- Atualizar `docs/coleta-de-dados.md` com os contratos atuais, a ordem de persistência/checkpoint, as fórmulas e a semântica de estados.
- Criar `docs/runbook-validacao-coleta.md` com pré-requisitos de VPN, seleção do evento de teste, renovação de sessão, execução prolongada, sinais de sucesso e diagnóstico de falhas.
- Atualizar `README.md` com os comandos oficiais de teste e validação.
- Registrar a solução do incidente em `docs/solutions/` depois que a implementação for comprovada, incluindo os endpoints válidos e os falsos positivos que devem ser evitados.

Escopo de interface

- Fazer uma revisão final do modal de coleta, lista de sites, gráficos e cards de VIP durante o teste prolongado.
- Confirmar que estados mudam em tempo real sem exigir reabrir o modal.
- Confirmar que um dado antigo continua visível com marca de desatualização, sem parecer atual.
- Confirmar que o painel volta a saudável somente depois de dados válidos ou de um vazio legítimo comprovado, conforme o coletor.

Escopo de testes

- Rodar toda a suíte offline.
- Rodar os testes marcados com VPN para o evento de integração configurado.
- Executar um teste prolongado mínimo de 30 minutos, cobrindo vários ciclos de KPI, VIP e alarmes.
- Durante o teste prolongado, forçar ao menos uma renovação de sessão e um reinício do aplicativo.
- Conferir diretamente no SQLite as chaves únicas, checkpoints monotônicos, ausência de duplicatas e continuidade dos timestamps.
- Manter os payloads de fixture sanitizados e versionados para que regressões possam ser reproduzidas sem VPN.

Como validar e testar

Gate automatizado offline

1. `python -m pytest -m "not vpn"` deve passar integralmente.
2. Nenhum teste de contrato pode aceitar vazio incondicionalmente quando o cenário fornece dados.
3. Fixtures 4G, 5G e VIP devem produzir valores e contagens determinísticos.

Gate com VPN

1. Rodar `python -m pytest tests/test_http_vpn.py -m vpn --vpn -v` com o evento de integração conhecido.
2. Rodar o validador por múltiplos ciclos para KPI e VIP.
3. Comparar pelo menos um timestamp de cada tecnologia e um trace de VIP com o iManager.
4. A task 5G real e a unidade dos contadores 5G são gates obrigatórios; sem elas, o suporte 5G permanece não liberado.

Gate visual e de duração

1. Manter o dashboard aberto por pelo menos 30 minutos com dados ativos.
2. Confirmar que os horários de tentativa, ciclo válido e dado avançam conforme seus significados.
3. Confirmar que a contagem do banco cresce somente com amostras novas.
4. Desconectar e reconectar a VPN; o painel deve sair de saudável, explicar a falha e recuperar após um ciclo real bem-sucedido.
5. Forçar renovação de sessão; nenhum módulo pode aparecer como saudável antes de sua chamada real passar.
6. Conferir que KPIs inválidos aparecem como lacunas/diagnóstico e que zeros reais continuam visíveis.

Sugestão de commit: `feat: adicionar validação operacional da coleta`

9. Ordem de execução e dependências

1. A fase 1 deve ser concluída primeiro. Sem ela, falhas das fases seguintes continuariam mascaradas como sucesso.
2. A fase 2 usa o resultado estruturado da fase 1 e cria a infraestrutura de checkpoints reutilizada pela fase 3.
3. A fase 3 depende da captura completa do fluxo da mesma task do VIP e reutiliza sessão, status e checkpoints das fases anteriores.
4. A fase 4 só começa depois que os testes offline das três fases anteriores passam.

Cada fase deve ser integrada separadamente. O sistema pode ser validado e revertido em limites claros, sem misturar a troca do contrato VIP com a implementação das fórmulas KPI.

10. Critérios finais de aceite

- Todos os KPIs listados nos dois arquivos de fórmulas possuem testes de cálculo e tratamento de invalidade.
- O Monitoring usa cursores retornados pelo servidor e não repete indefinidamente o histórico.
- Células 4G e 5G são associadas à task e ao objeto corretos; lacunas de configuração aparecem como parcial.
- KPIs e traces reenviados não criam duplicatas.
- O VIP usa o fluxo incremental confirmado no navegador para a mesma task configurada.
- Uma renovação só é bem-sucedida depois que a operação original funciona.
- O agendador nunca atualiza sucesso após erro capturado ou retorno vazio causado por falha.
- A interface diferencia dado novo, vazio legítimo, parcial, erro, autenticação e desatualização.
- A suíte offline, os testes com VPN e o teste prolongado passam.
- Os payloads de teste não contêm cookies, tokens, IMSI ou outros dados sensíveis.
- A documentação descreve o contrato realmente implementado e o procedimento de validação.

11. Riscos que não podem ser tratados como detalhes

- **Task PM 5G desconhecida:** sem uma resposta real 5G não é possível validar nomes de objetos, unidades e counters disponíveis. Isso bloqueia a liberação 5G, não a criação dos testes de fórmula.
- **Captura VIP incompleta e de outra task:** a inicialização e o decode devem ser capturados para a task correta antes da troca definitiva do fluxo.
- **Unidades 5G não declaradas nas fórmulas:** devem ser verificadas contra o iManager; não aplicar fator de conversão por analogia com 4G.
- **Migração de bancos existentes:** índices únicos precisam remover apenas duplicatas comprovadas e preservar registros históricos legítimos.
- **Sessão compartilhada:** Monitoring e Alarmes usam a mesma área de autenticação, mas uma operação válida não prova automaticamente que a outra está saudável.
- **Backlog de trace:** limites de lote não podem avançar o cursor além de mensagens ainda não decodificadas.